

Deep Learning

COSC 2753 | Semester 2, 2026

Machine Learning

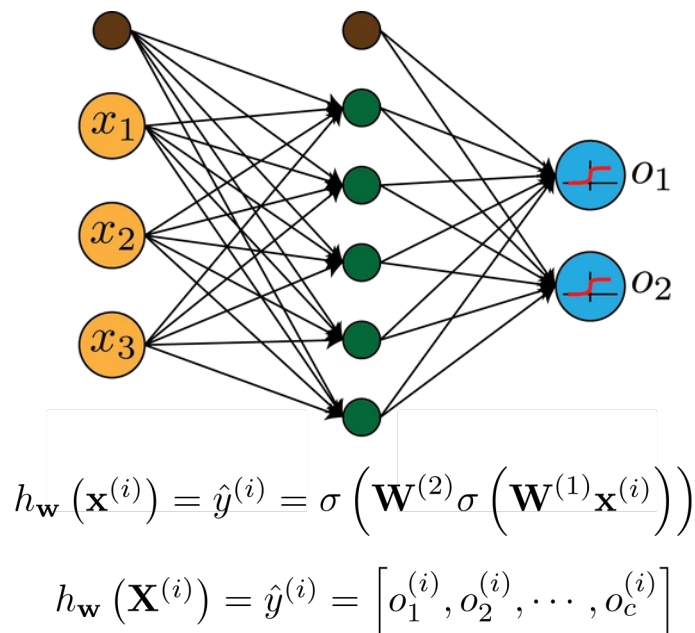


Content



- Computer vision
- Introduction to Deep learning
- Basics MLP to CNN
- CNN Architectures
- Practical Aspects

Multi-Layer Perceptron (MLP)



Experience: $D = \{X(i), y(i)\}$

Hypothesis space: **Non-linear when activation is non-linear.**

Loss: Log-loss (binary/categorical cross-entropy)

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\sum_{j=0}^c y_j^{(i)} \log(\hat{y}_j^{(i)}) = -\sum_{j=0}^c y_j^{(i)} \log(o_j^{(i)})$$

Optimization: Back propagation (stochastic gradient decent)

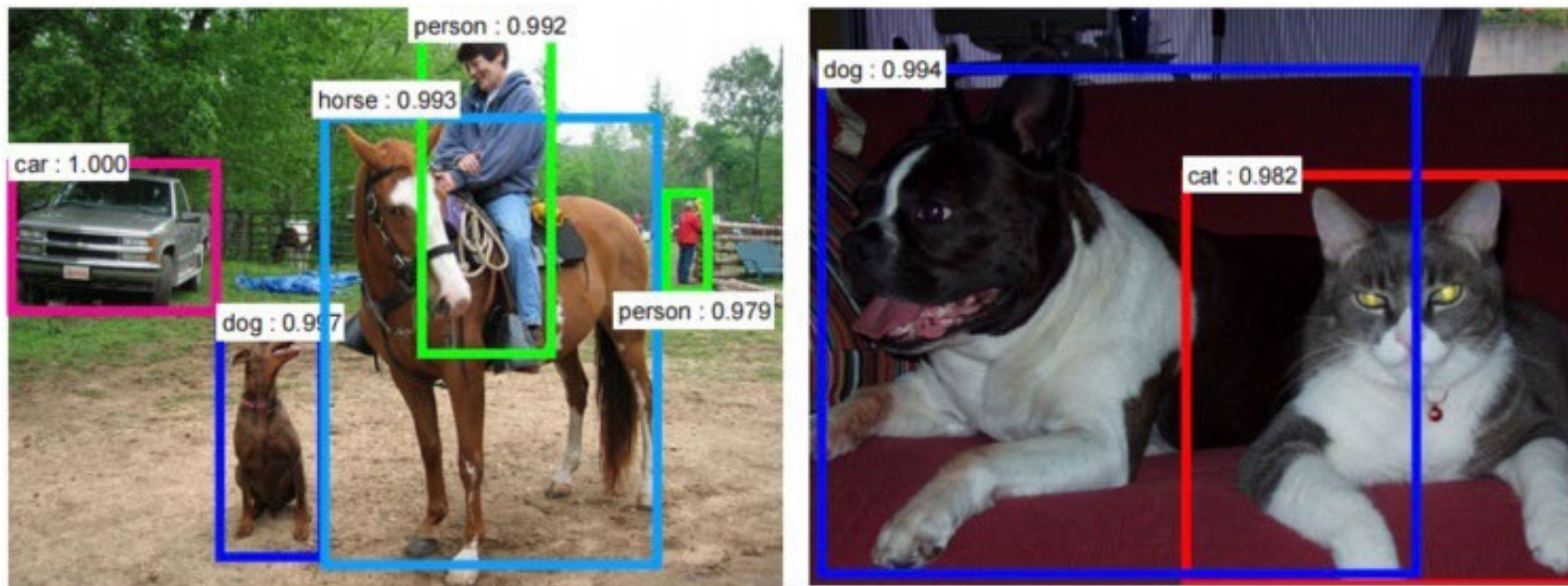
This approach gives up on the convexity of the training problem, but the benefits outweigh the harms.
Ian Good fellow, Deep learning book.



Computer Vision

Convolution Feature extraction

Object Detection & Recognition



Source: <https://arxiv.org/pdf/1506.01497v3.pdf>

Image Captioning



a soccer player is kicking a soccer ball



a street sign on a pole in front of a building



a couple of giraffe standing next to each other

Source: <https://cs.stanford.edu/people/karpathy/neuraltalk2/demo.html>

Fashion Image Captioning

Fashion image	Ground truth captions	Spatial-wise attention	Our proposed method
	Long sleeve supple leather jacket in black, notched lapel collar, off set zip closure at front, zippered pocket at body, epaulet, zippered expansionpanel at cuff, welt pocket at fully lined interior, silver tone hardware, tone stitching.	Long sleeve French terry hoodie in black, draw string at hood, logo printed in white at chest, kangaroo pocket at waist, rib knit cuff and hem, tonal stitching.	Long sleeve grained leather jacket in black, notched lapel collar with press stud fastening, off set zip closure at front, zipped pocket at body, zippered expansion panel at cuff, zippered pocket at fully lined interior, silver tone hard-ware, tonal stitching.
	Long sleeve blazer in tone of blue, abstract pattern printed throughout, the three-button closure at front, notched lapel collar, four button surgeon cut,belt pocket at front and at fullylined interior, single vent at back, tonal stitching.	Long sleeve quilted down filled jacket in black, multi color graphic printed throughout, rib knit stand collar, cuff and hem, two-way zip closure at front, welt pocket at waist, utility pocket at upper sleeve, weltpocket at fully lined interior, silver tone hair wear, tonalstitching, brushed 90% goose down, 10% feather.	Long sleeve shirt in tone of blue, blue and white, button closure at front, single button barrel cuff, tonal stitching.

Source: Bao Nguyen et. al. *Channel and spatial attention mechanism for fashion image captioning*,

DOI: [10.11591/ijece.v13i5.pp5833-5842](https://doi.org/10.11591/ijece.v13i5.pp5833-5842)



Fashion Item Matching



(a) Casual



(b) Weekend



(c) Cocktail party



(d) Wedding guest



(e) Work



(f) School



(g) Dating

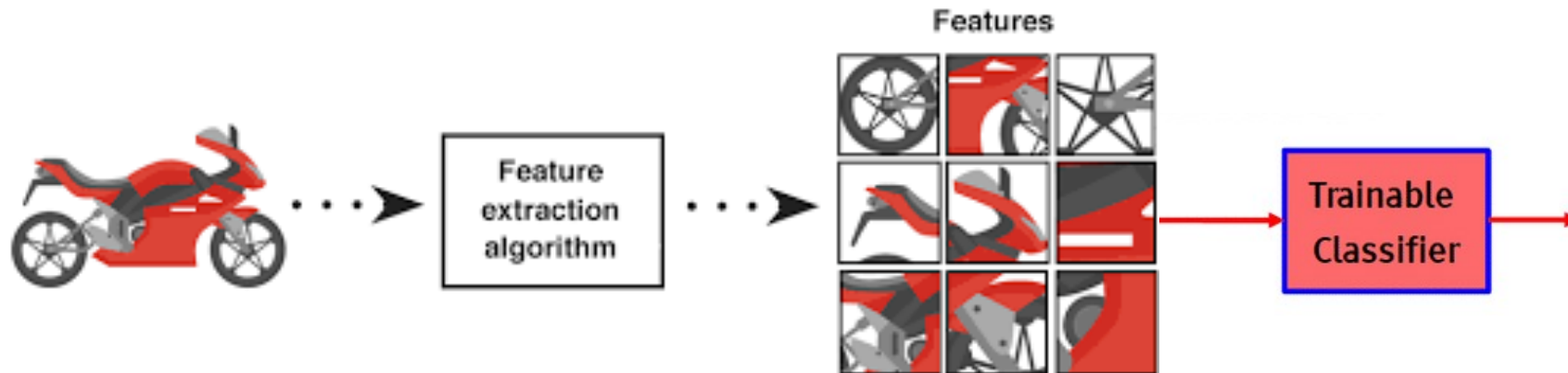


(h) Party

Source: Bao Nguyen et. al. *An efficient framework for outfit compatibility prediction towards occasion*

DOI: [10.1007/s00521-023-08431-1](https://doi.org/10.1007/s00521-023-08431-1)

Computer Vision



Feature Extraction:

- Discard redundant information in an image
- Reduce the dimensionality (size) of the image to a manageable scale
- Transform the image into a subset of important or “informative” features
- Use these new features to do actual image recognition

Shape-Based Feature Extraction

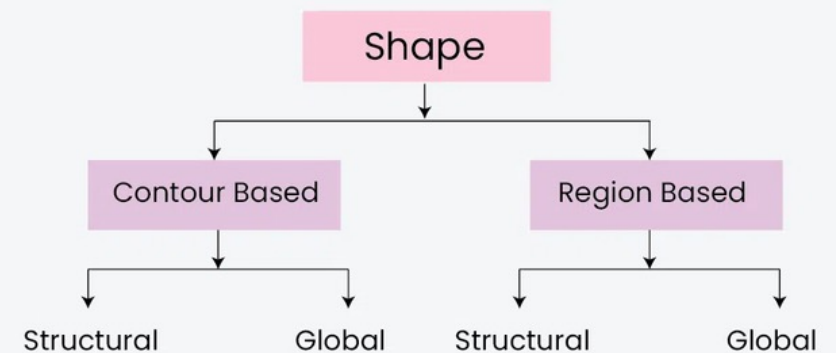
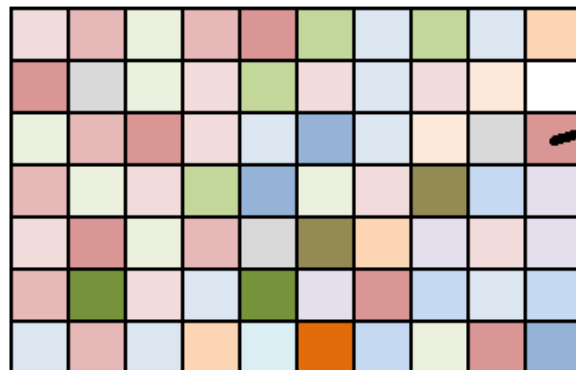
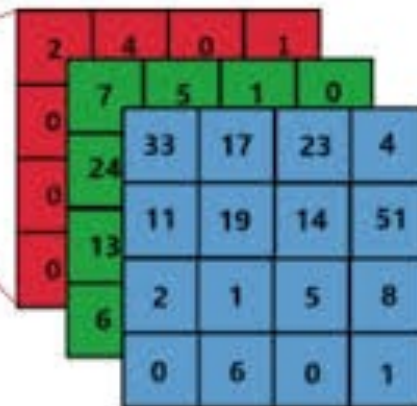


Image representation



RGB (218, 150, 149)

R = 11011010
G = 10010110
B = 10010101

	Red	Green	Blue	Hexadecimal code
	0	0	0	#000000
	255	255	255	#FFFFFF
	255	0	0	#FF0000
	0	255	0	#00FF00
	0	0	255	#0000FF
	255	128	0	#FF8000
	255	255	0	#FFFF00
	128	128	128	#808080

Traditional Computer Vision Pipeline



Before conducting any image recognition/detection a traditional processing pipeline might consist of a variety of steps such as:

- Greyscale conversion
- Normalisation
- Gaussian filtering
- Downsampling
- Edge Detection
- Segmentation



Image Features

Histogram of Gradient HoG

<https://www.learnopencv.com/histogram-of-oriented-gradients/>

Input image



Histogram of Oriented Gradients

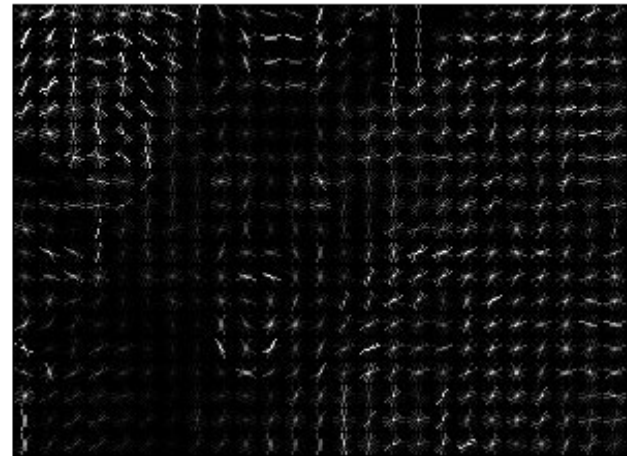


Image Shape: (306, 410, 3) Flattened shape : 376380
HoG feature Shape: 3800

More types of feature extractors: <https://scikit-image.org/docs/dev/api/skimage.feature.html>

Image Filters



1	0	-1			
[0.8	0.74	0.6	0.5	0.47	0.55
2	0	-2			
[0.81	0.73	0.58	0.45	0.41	0.46
1	0	-1			
[0.81	0.73	0.58	0.43	0.35	0.38
[0.81	0.73	0.58	0.43	0.31	0.31
[0.82	0.73	0.61	0.44	0.29	0.27
[0.83	0.75	0.62	0.44	0.27	0.25

*

1	0	-1
2	0	-2
1	0	-1

Sobel Filter
(Kernel)



	-0.89				



Image Filters



[0.8	¹ 0.74	⁰ 0.6	⁻¹ 0.5	0.47	0.55
[0.81	² 0.73	⁰ 0.58	⁻² 0.45	0.41	0.46
[0.81	¹ 0.73	⁰ 0.58	⁻¹ 0.43	0.35	0.38
[0.81	0.73	0.58	0.43	0.31	0.31
[0.82	0.73	0.61	0.44	0.29	0.27
[0.83	0.75	0.62	0.44	0.27	0.25

*

1	0	-1
2	0	-2
1	0	-1

Sobel Filter
(Kernel)



	-0.89	-1.09			



Image Filters



0.8	0.74	0.6	0.5	0.47	0.55
0.81	0.73	0.58	0.45	0.41	0.46
0.81	0.73	0.58	0.43	0.35	0.38
0.81	0.73	0.58	0.43	0.31	0.31
0.82	0.73	0.61	0.44	0.29	0.27
0.83	0.75	0.62	0.44	0.27	0.25

*

1	0	-1
2	0	-2
1	0	-1

Sobel Filter
(Kernel)



	-0.89	-1.09	-0.71	0.02	
	-0.92	-1.18	-0.91	-0.2	
	-0.9	-1.2	-1.11	-0.44	
	-0.87	-1.21	-1.26	-0.63	

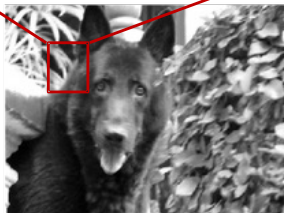


Image Filters



0.8	0.74	0.6	0.5	0.47	0.55
0.81	0.73	0.58	0.45	0.41	0.46
0.81	0.73	0.58	0.43	0.35	0.38
0.81	0.73	0.58	¹ 0.43	⁰ 0.31	⁻¹ 0.31
0.82	0.73	0.61	² 0.44	⁰ 0.29	⁻² 0.27
0.83	0.75	0.62	¹ 0.44	⁰ 0.27	⁻¹ 0.25

*

1	0	-1
2	0	-2
1	0	-1



		-0.89	-1.09	-0.71	0.02
		-0.92	-1.18	-0.91	-0.2
		-0.9	-1.2	-1.11	-0.44
		-0.87	-1.21	-1.26	-0.63

Sobel Filter
(Kernel)

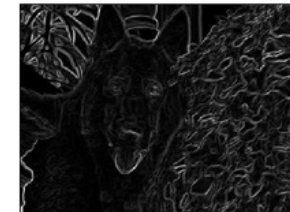


Image Filters

0	0	0	0	0	0	0	0
0	0.8	0.74	0.6	0.5	0.47	0.55	0
0	0.81	0.73	0.58	0.45	0.41	0.46	0
0	0.81	0.73	0.58	0.43	0.35	0.38	0
0	0.81	0.73	0.58	0.43	0.31	0.31	0
0	0.82	0.73	0.61	0.44	0.29	0.27	0
0	0.83	0.75	0.62	0.44	0.27	0.25	0
0	0	0	0	0	0	0	0

Padding

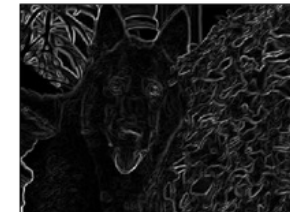
*

1	0	-1
2	0	-2
1	0	-1

Sobel Filter
(Kernel)



2.2	-0.62	-0.75	-0.43	0.11	-1.36
2.92	-0.89	-1.09	-0.71	0.02	-1.64
2.92	-0.92	-1.18	-0.91	-0.2	-1.41
2.93	-0.9	-1.2	-1.11	-0.44	-1.25
2.95	-0.87	-1.21	-1.26	-0.63	-1.16
2.24	-0.64	-0.93	-1.01	-0.54	-0.84



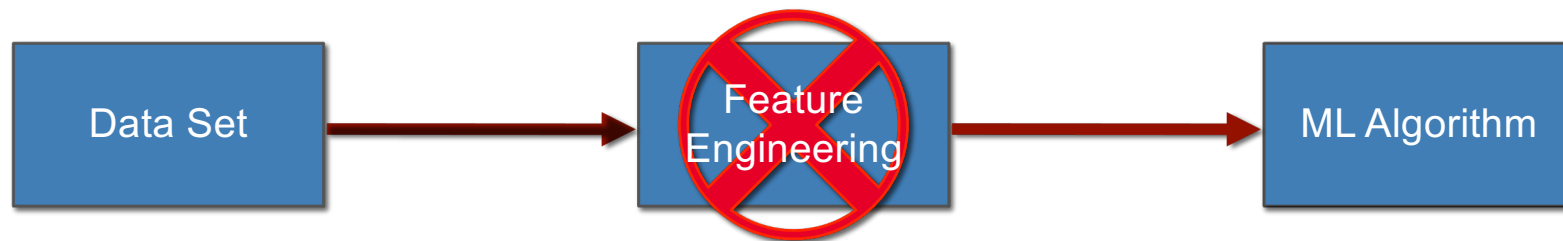
Traditional ML Limitations

Up until the success of Deep Neural Networks, ML is significantly limited by:

Features/Attributes of the Data set

Size of the Data set

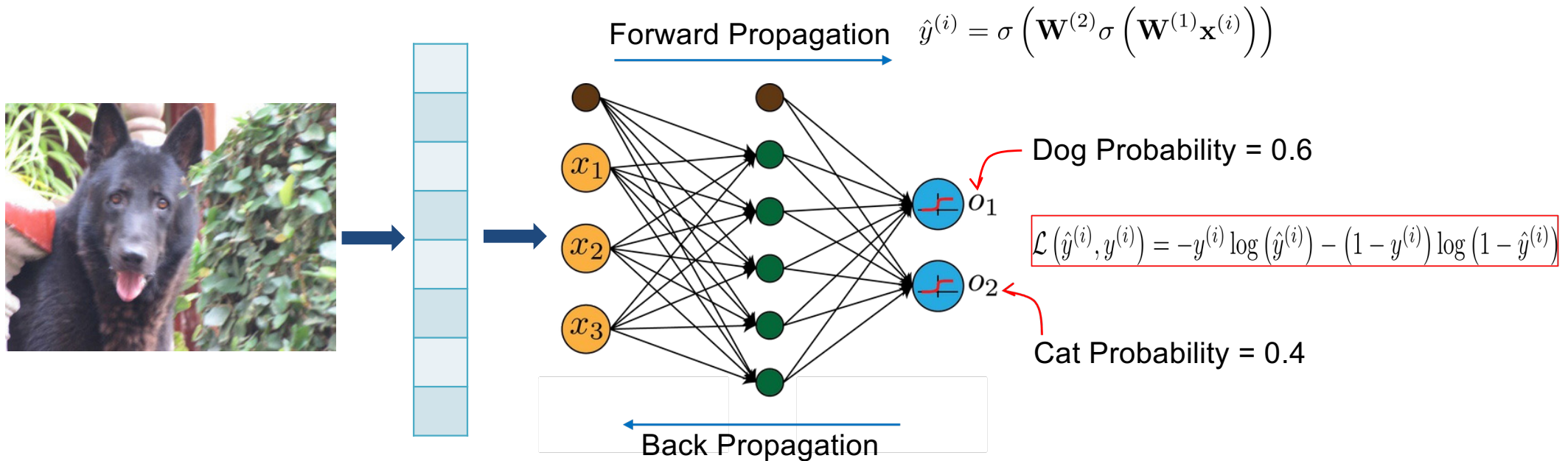
Goal: Eliminate (or automate) the feature engineering



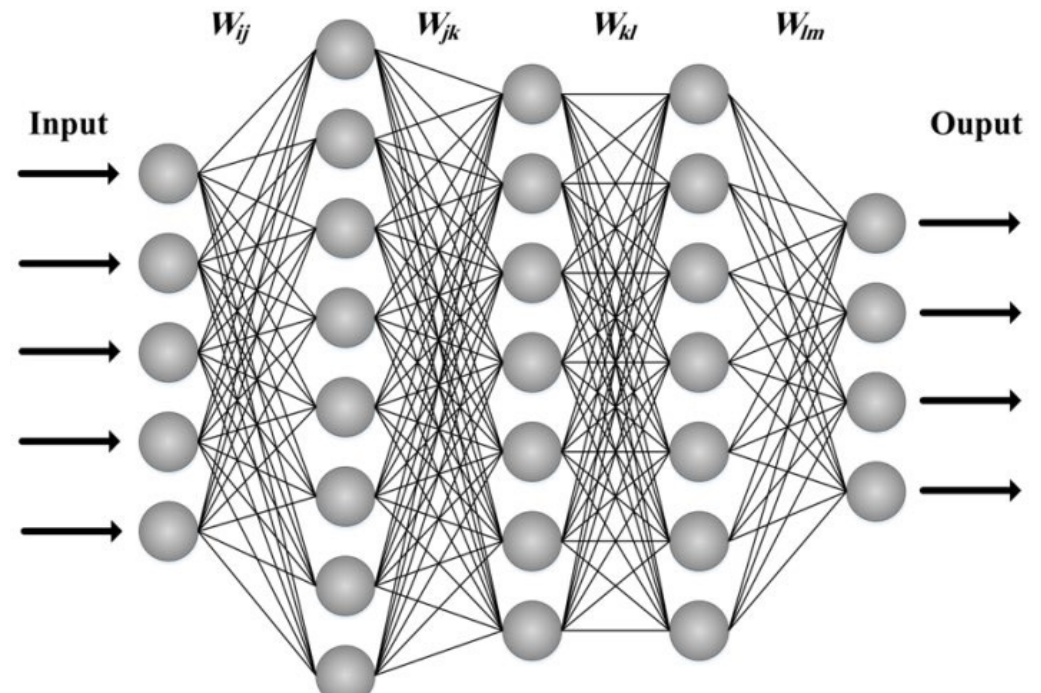


MLP to CNN

Multi-Layer Perceptron (MLP)



MLP for complex Tasks



$$256 \times 256 = 65,536$$

Limitations with MLP approach



- Model complexity increase with input size.
 - Still require to construct some features by hand.
- Stacking large number of hidden layers will result in
 - Diminishing gradient.
- Large connections of the nodes that are unnecessary.
- Treat each pixel in image independently
 - loose some spatial information (important patterns/structures)

Does not scale well with the complexity of task.

Deep Neural Networks



A ***deep neutral network***: a neural network with a large number of hidden layers

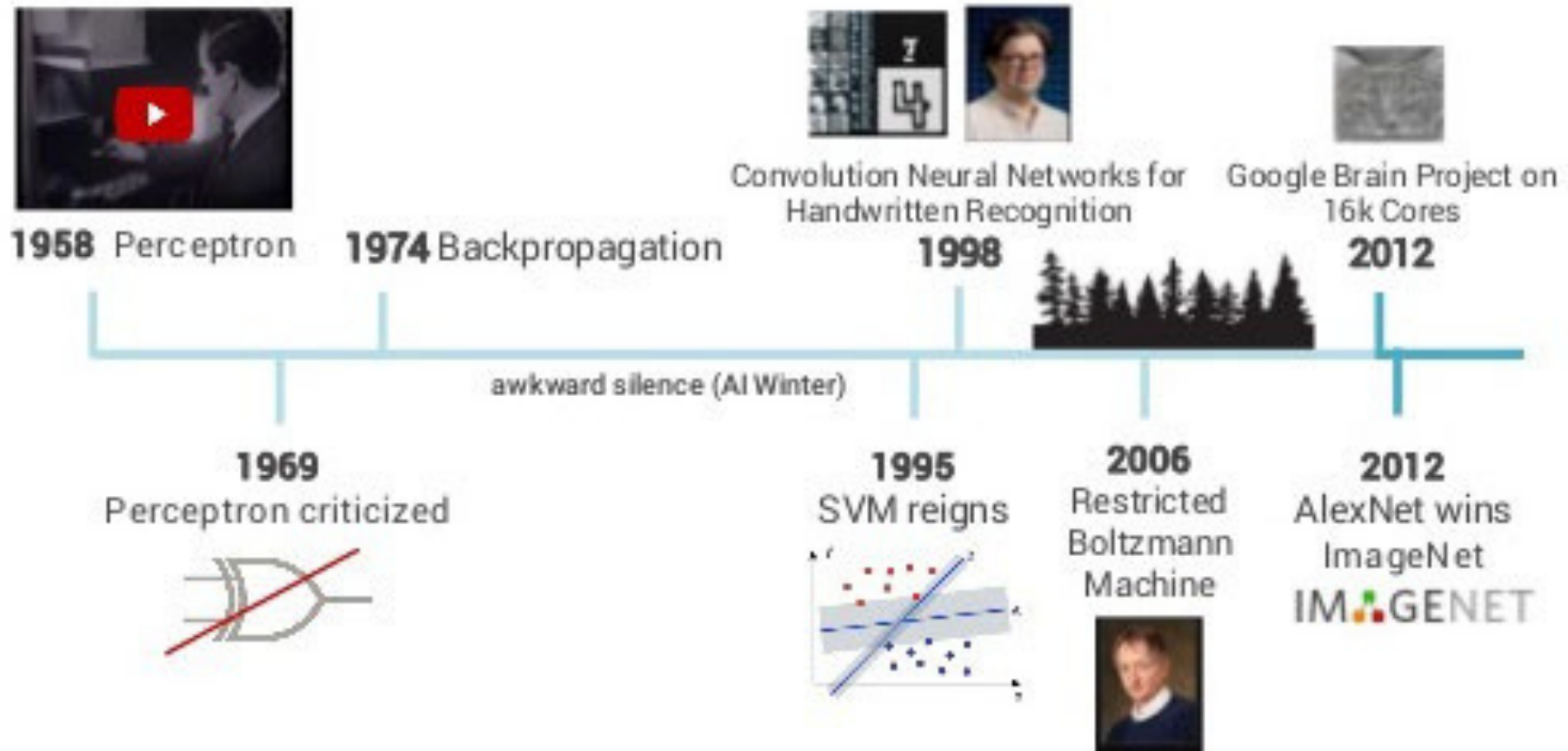
There is a variety of different deep net structures

One widely used deep net structure is a ***Convolution Neural Network (CNN)***

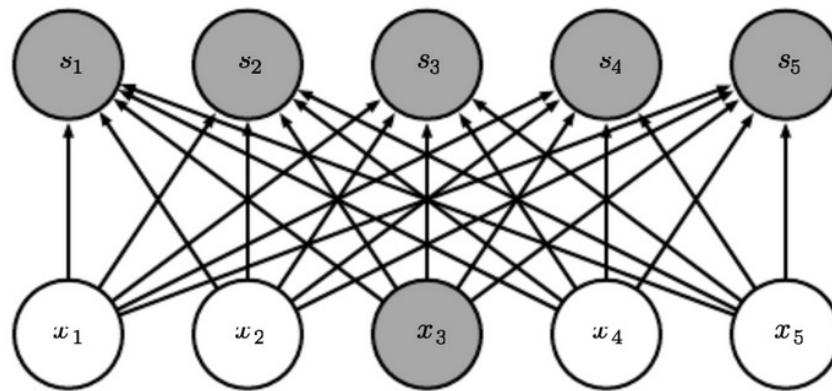
At the moment, this is what most people refer to when discussing deep learning

Very successful for image-based, vision ML tasks

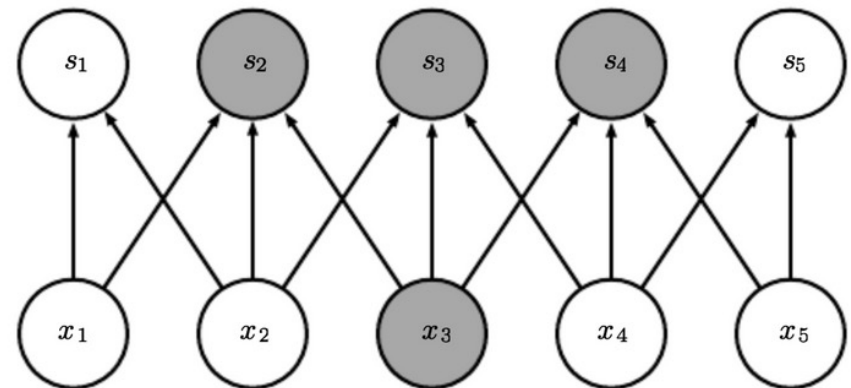
History to Deep Nets



Sparse Connectivity



Fully connected



Sparse connected

Image: Deep learning, Ian Goodfellow

Learning Convolution Filters



0.8	0.74	0.6	0.5	0.47	0.55
0.81	0.73	0.58	0.45	0.41	0.46
0.81	0.73	0.58	0.43	0.35	0.38
0.81	0.73	0.58	0.43	0.31	0.31
0.82	0.73	0.61	0.44	0.29	0.27
0.83	0.75	0.62	0.44	0.27	0.25



*

$w_{1,1}$	$w_{1,2}$	$w_{1,3}$
$w_{2,1}$	$w_{2,2}$	$w_{2,3}$
$w_{3,1}$	$w_{3,2}$	$w_{3,3}$

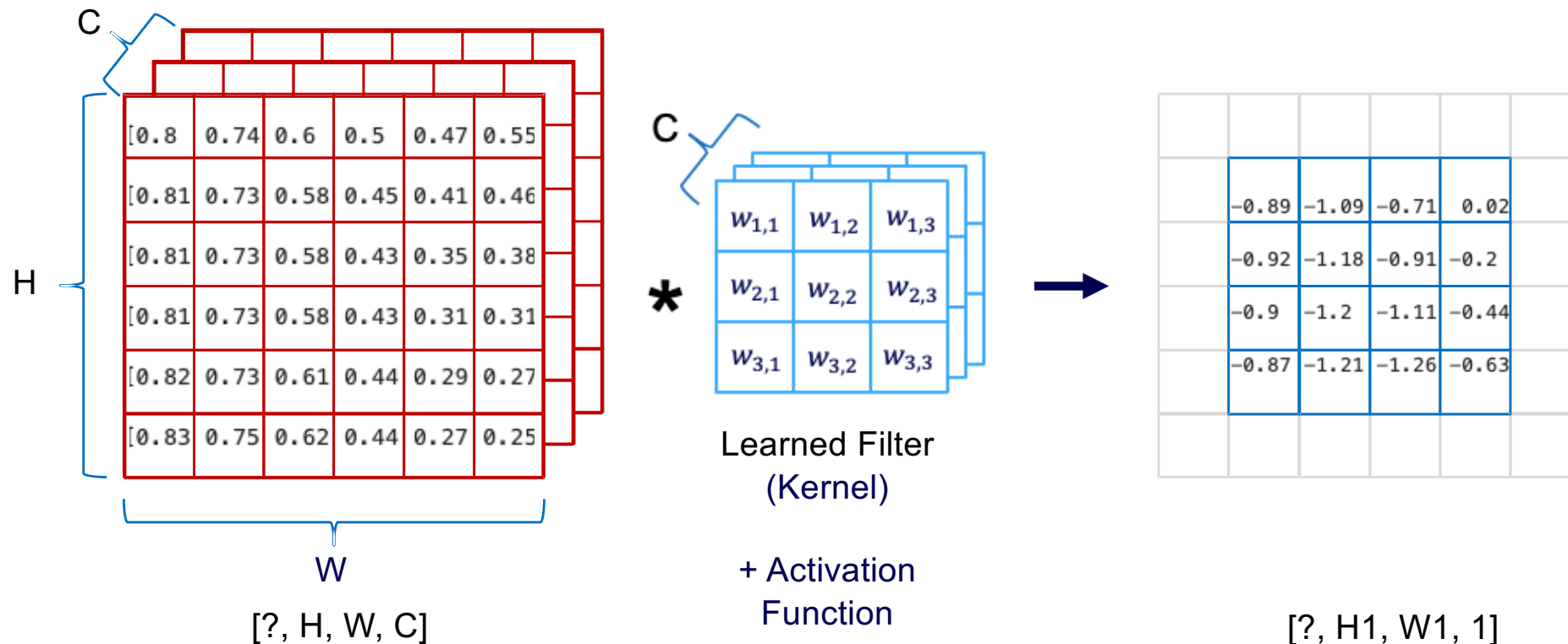
Learned Filter
(Kernel)

+ Activation
Function



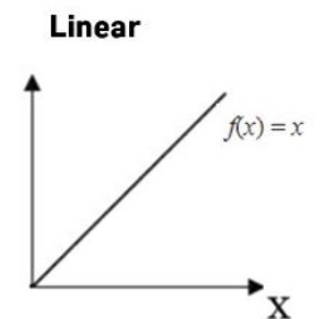
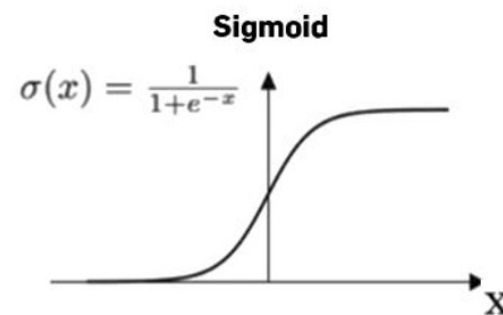
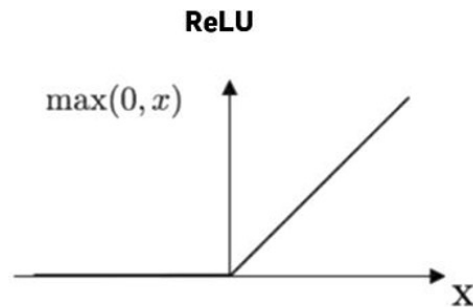
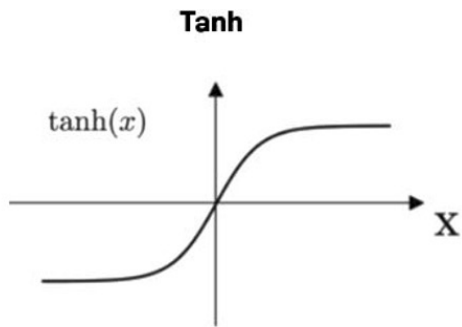
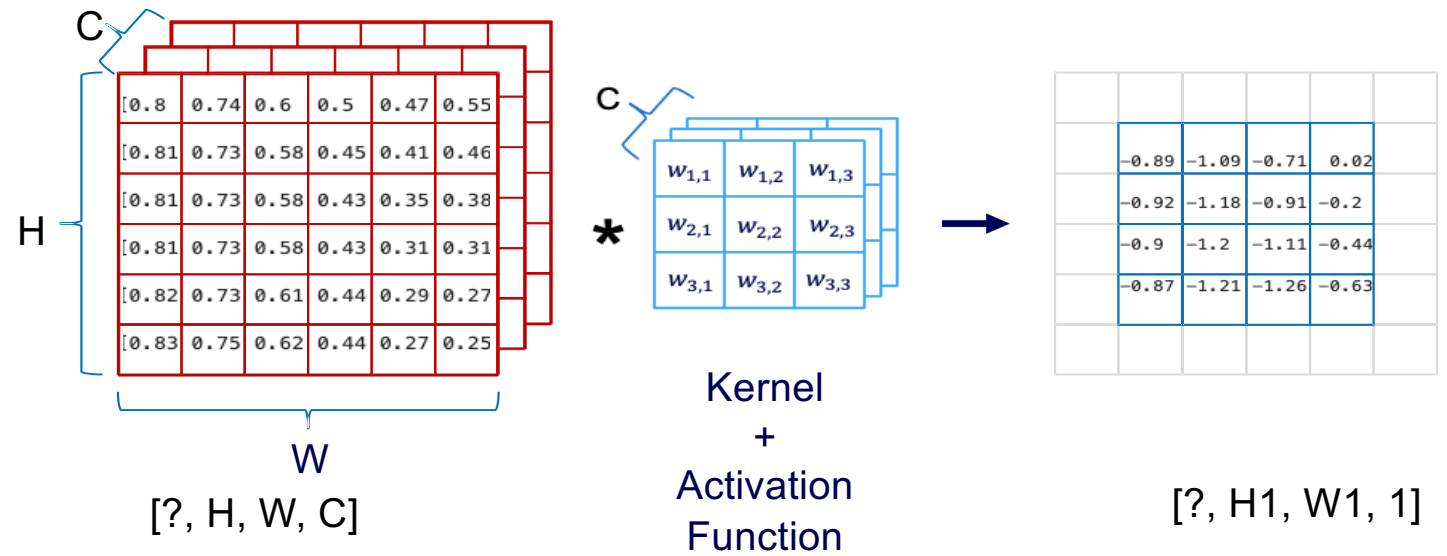
	-0.89	-1.09	-0.71	0.02	
	-0.92	-1.18	-0.91	-0.2	
	-0.9	-1.2	-1.11	-0.44	
	-0.87	-1.21	-1.26	-0.63	

Learning Convolution Filters

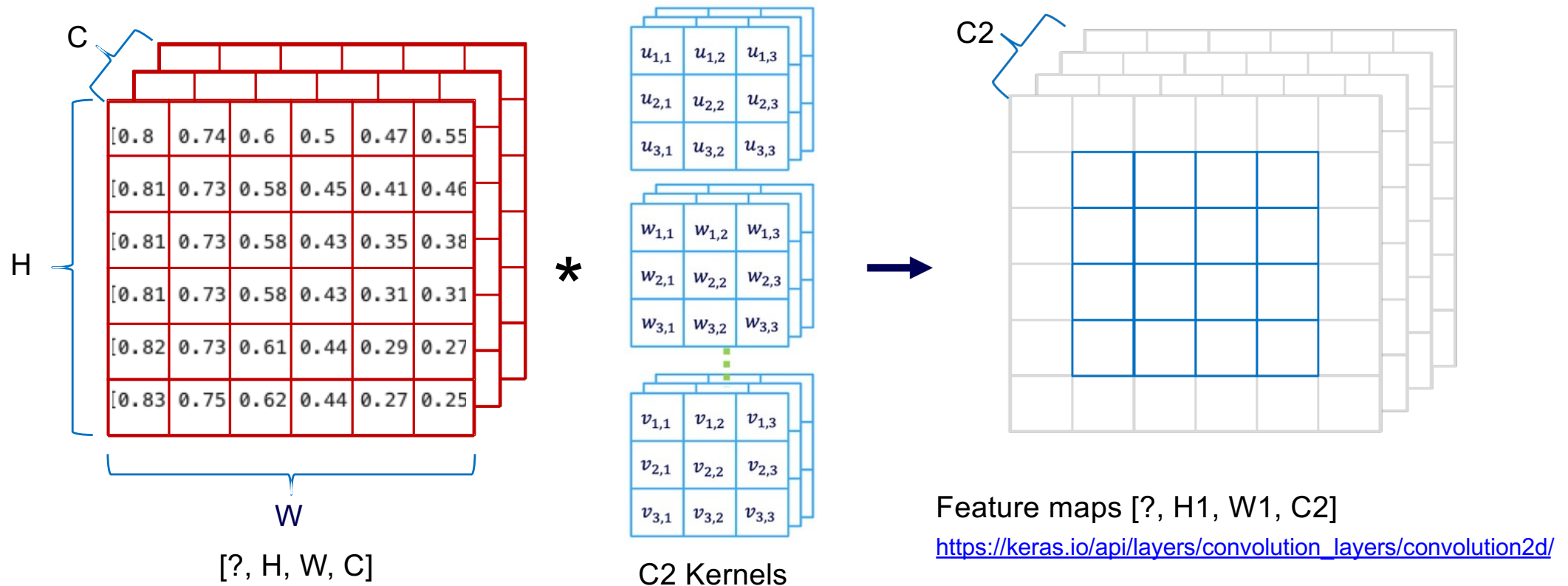


?: the batch size
one Kernel $\rightarrow$ one feature

Activation functions

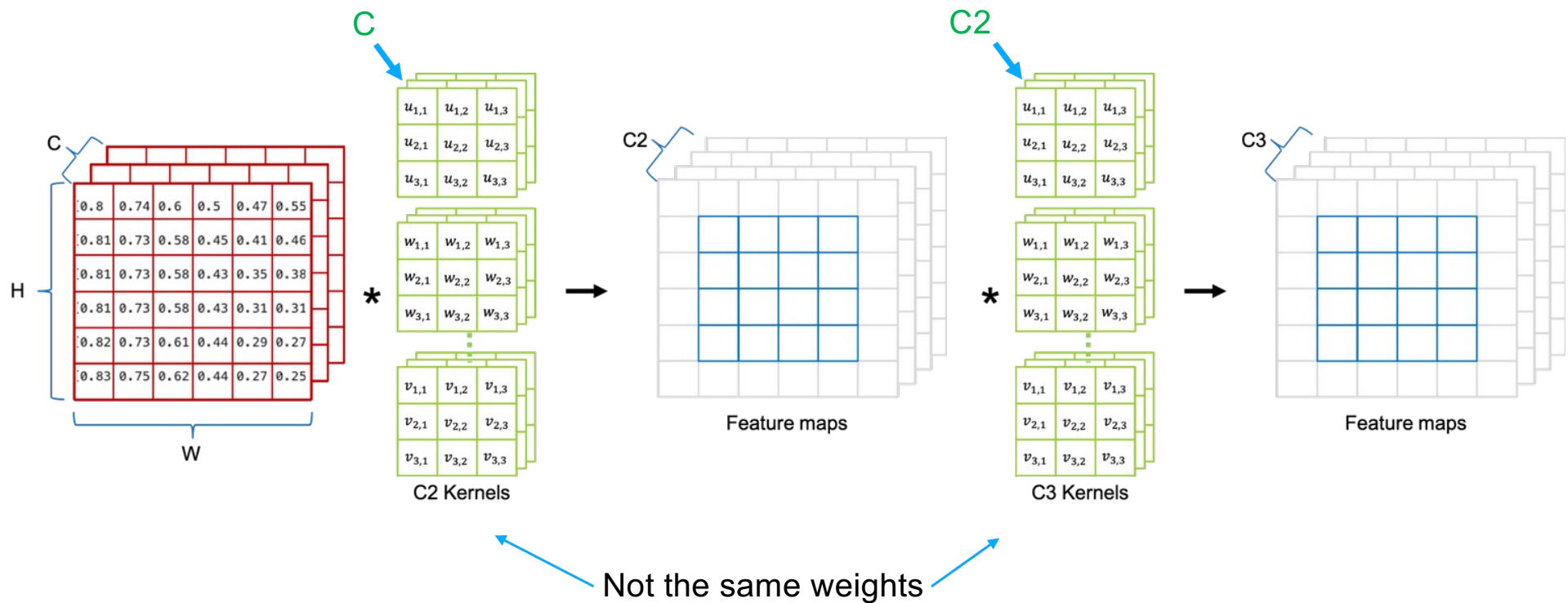


Learning Convolution Filters

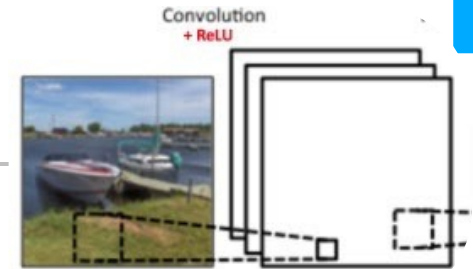


Learning Convolution Filters

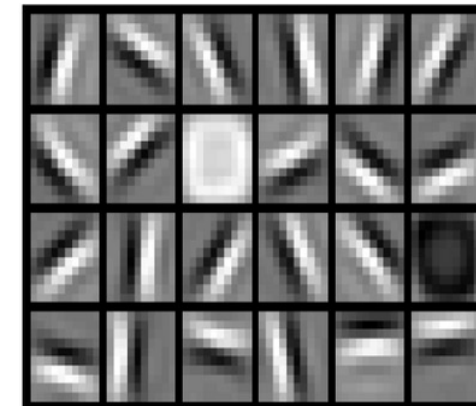
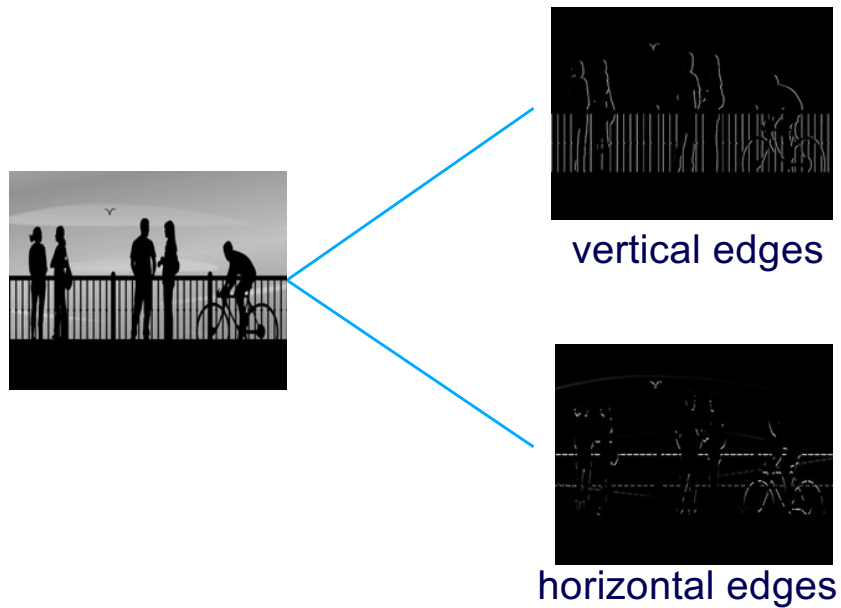
Stacked Convolution layers - Activations



Convolutional Layer



After training, what does a convolutional layer look like?



Learned Feature

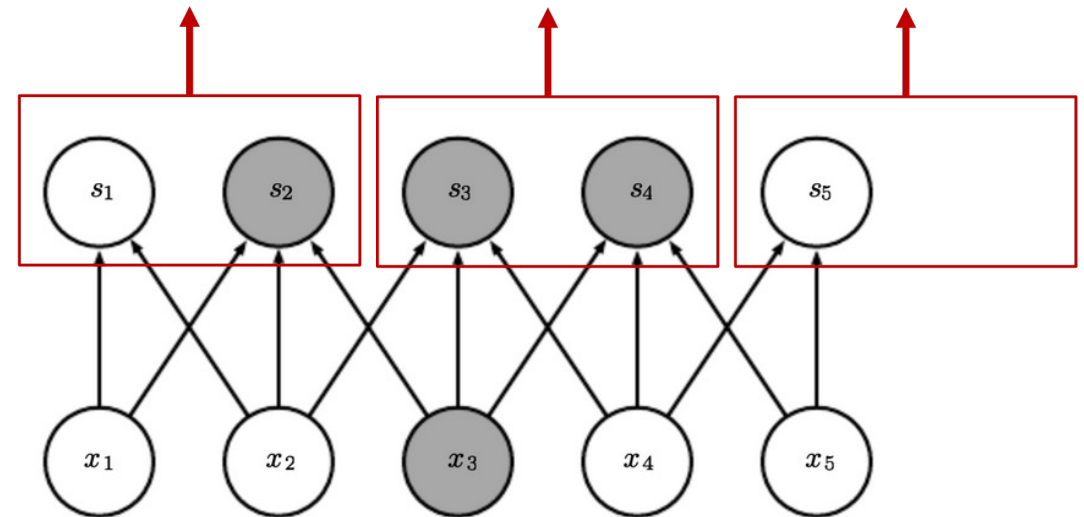
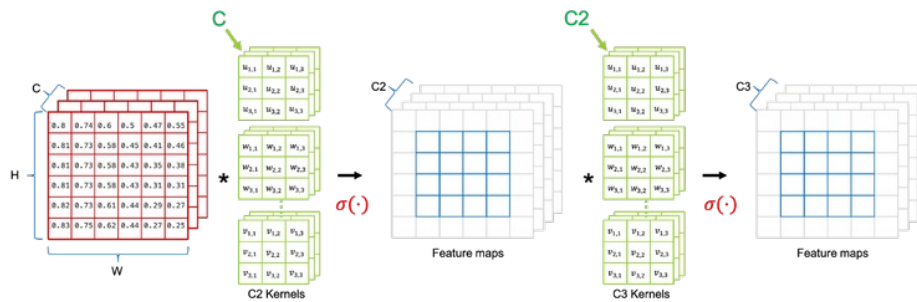
CNN Building-Blocks



04 main building blocks of CNNs

- Convolutions (2D)
- Non-Linearity (Activations)
- Pooling (or sub-sampling)
- Fully Connected MLP

Pooling



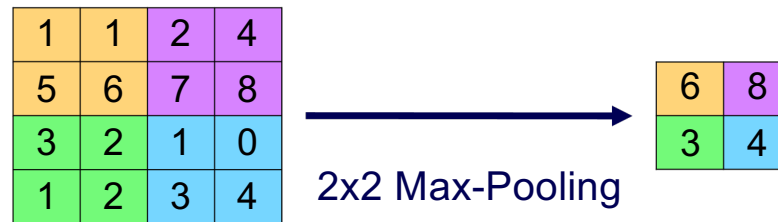
Pooling

Pooling is sub-sampling:

Reduces the dimensionality of each feature map

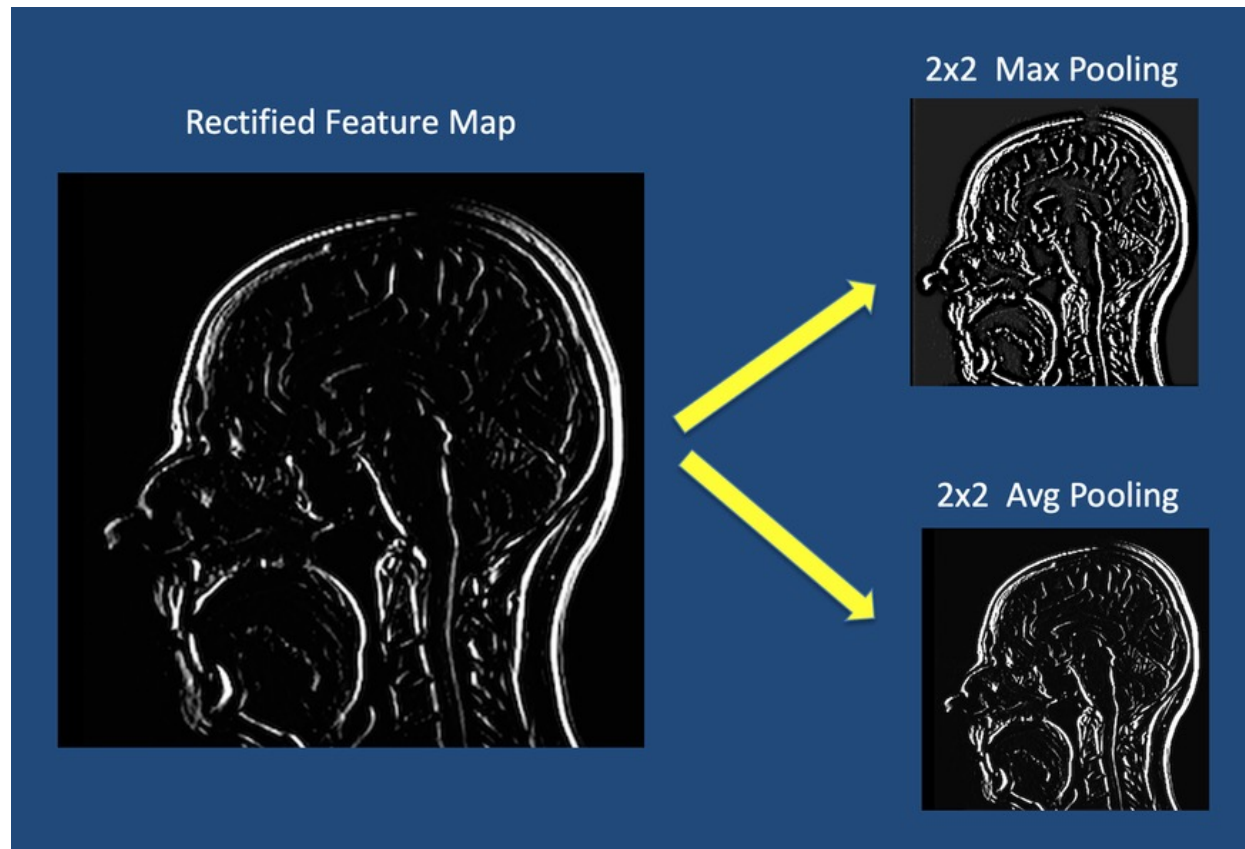
Applied to each feature map independently

Pooling is one of three typical simple matrix operations: Max, Average, Sum



Pooling Step

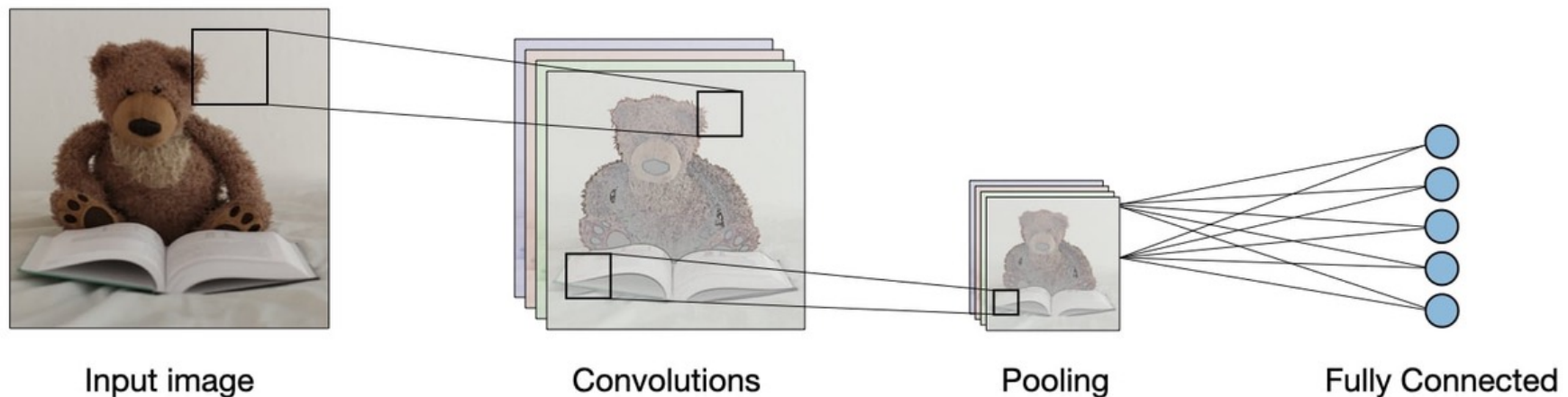
Example of Pooling on a rectified featuremap



Pooling

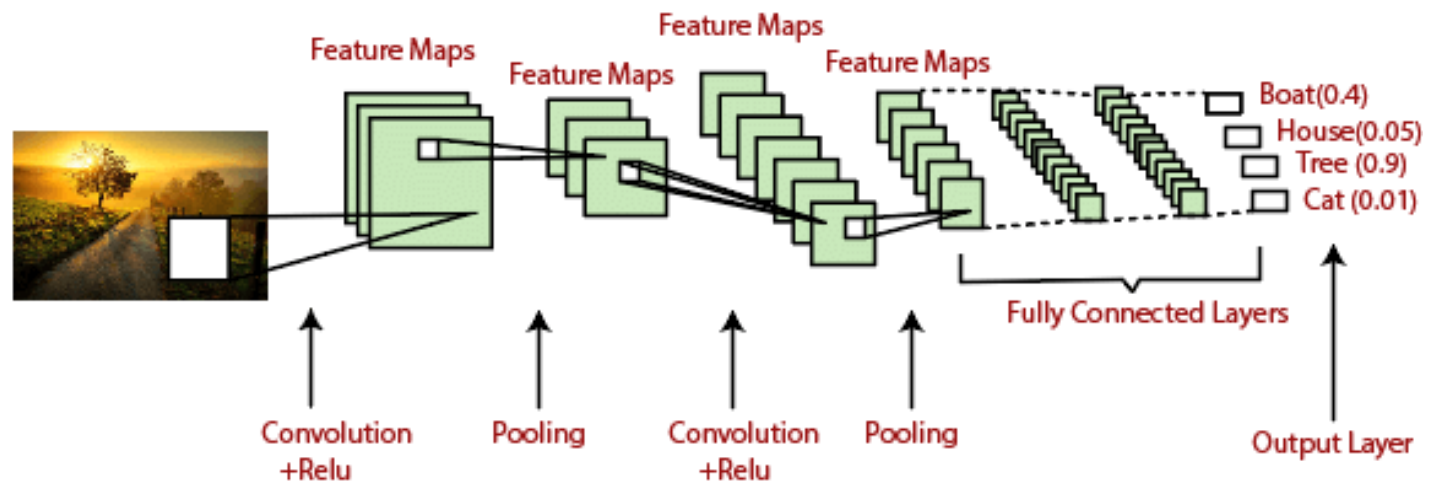
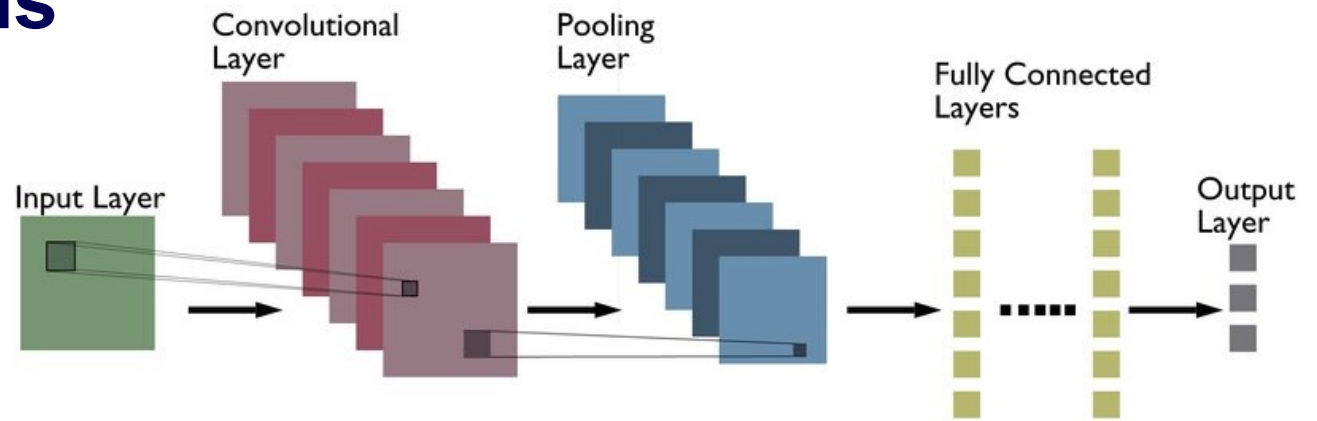
The purpose of Pooling is:

- Make the input representations (feature dimension) smaller and more manageable
- Reduce the number of parameters in the network, helping to limit overfitting
- Invariant to small transformations, scaling distortions and translations of image



Repeated Convolutions & Pooling

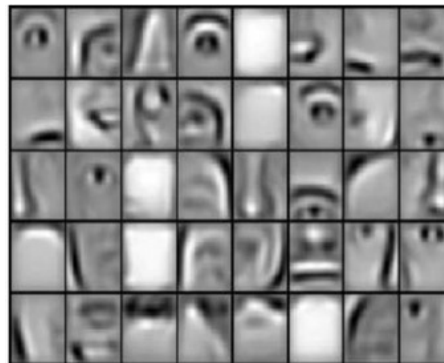
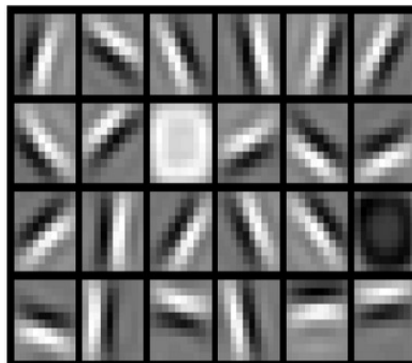
- Different pooling methods may also be used
- Pooling may be discarded for later convolutions



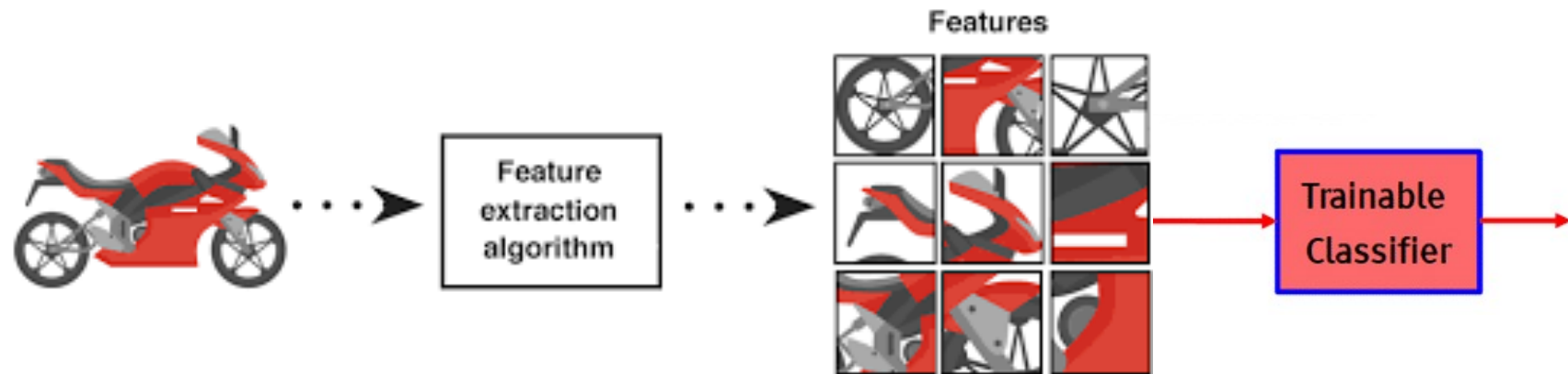
Repeated Convolutions

Multiple convolutional layers effectively construct more complex feature maps

These feature maps are often what people refer to when they discuss what a CNN (or deep net) has learnt/discovered.



Repeated Covolutions vs Computer Vision



 **Deep Learning:** Representations are hierarchical and trained



CNN Building-Blocks



04 main building blocks of CNNs

- Convolutions (2D)
- Non-Linearity (Activations)
- Pooling (or sub-sampling)
- Fully Connected MLP

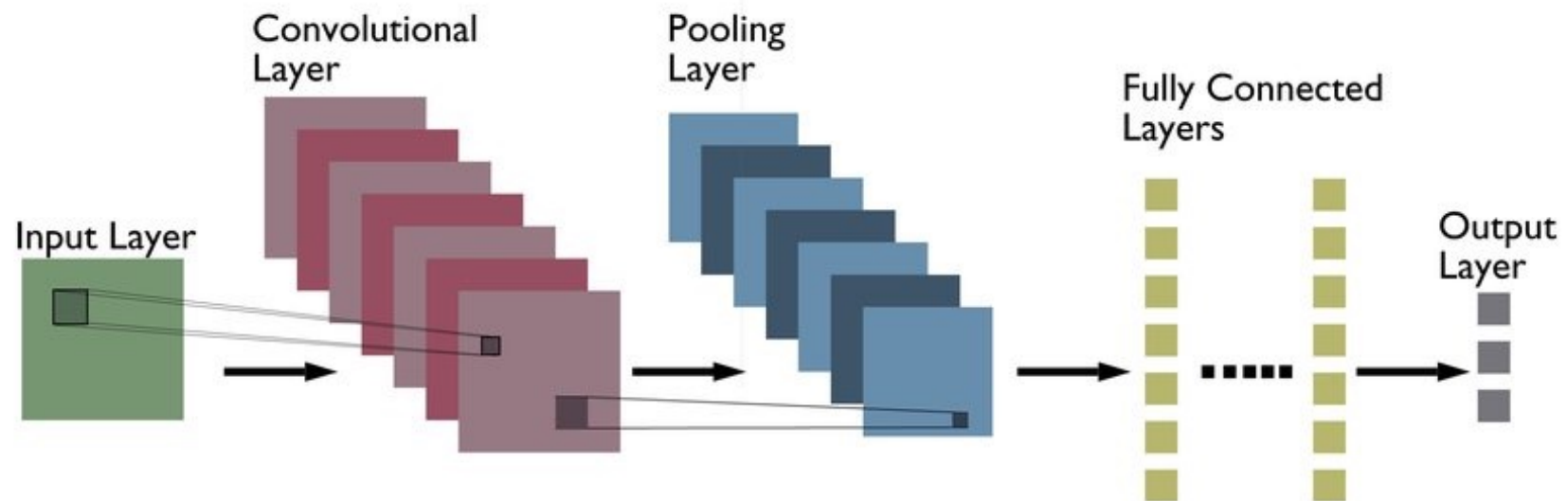
Fully Connected Layer

The last stage of a CNN is a traditional fully connected layer.

That is, an MLP For classification a **soft max activation** function in the output layer.

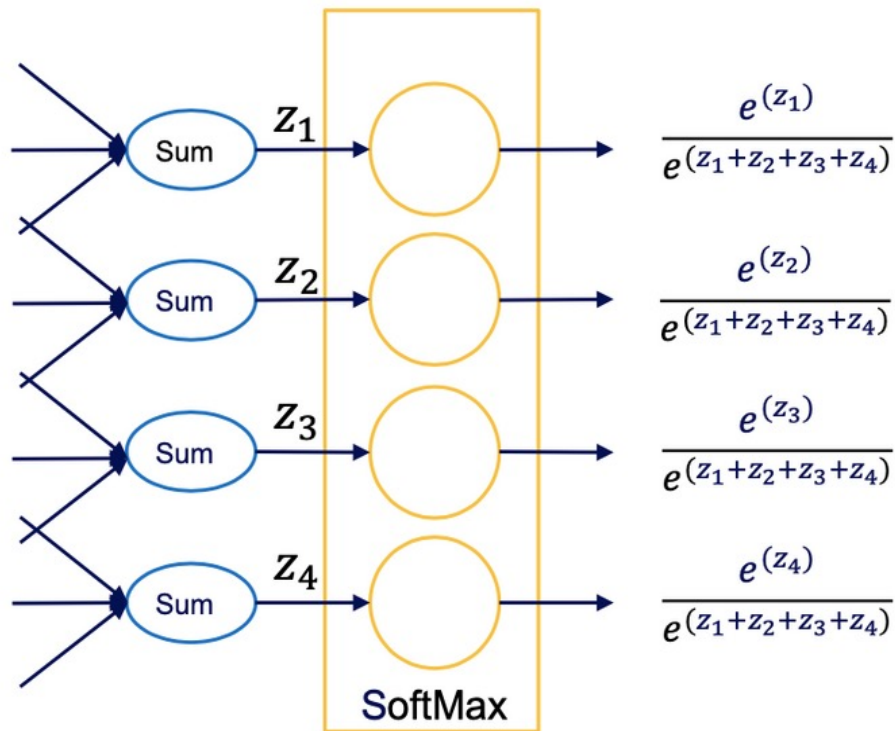
The number of layers in the MLP may be varied

Typically between 2-3 layers are used

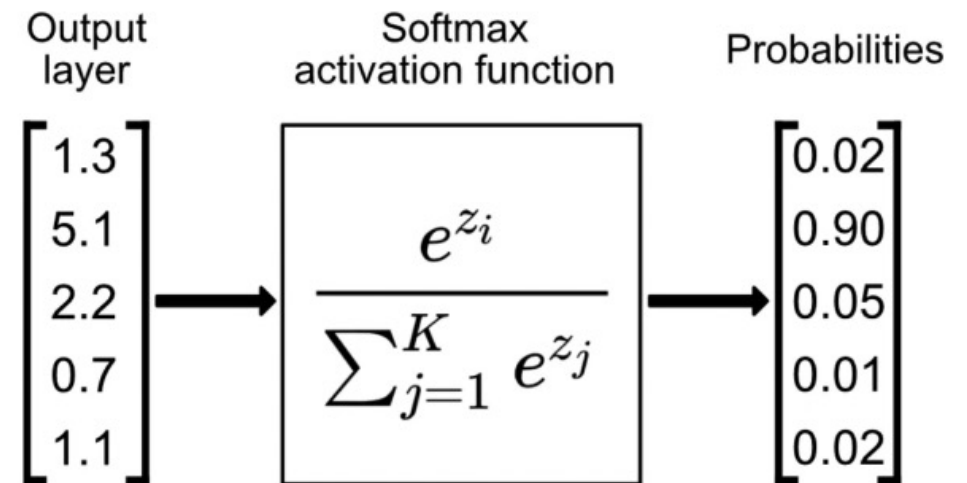


Softmax

Typically SoftMax activation is used at the output layer for classification



$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

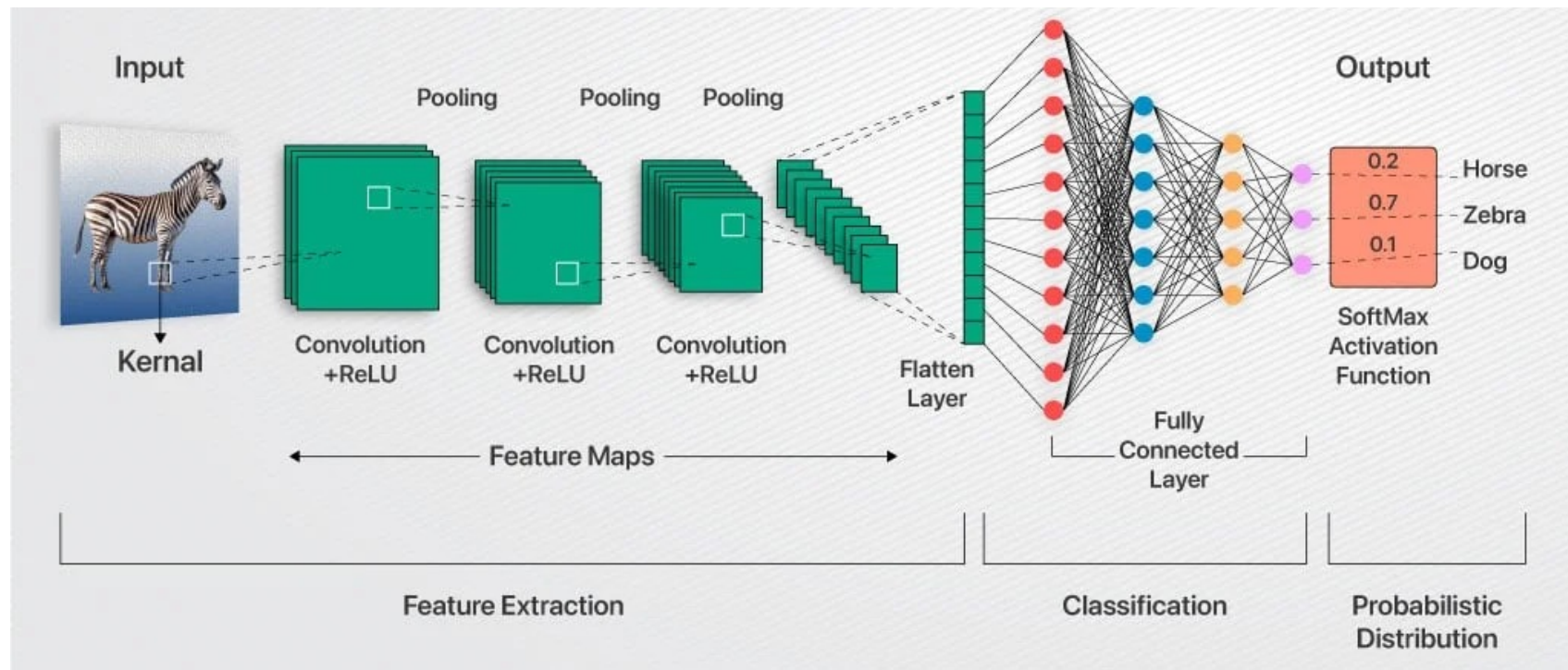


Final CNN

The final CNN structure effectively gives two parts:

- The convolution: **feature construction** to transform the input into a suitable attribute space
- **Classification**: using the constructed/learnt features for classifying

Thus a CNN learns both classification and feature construction



Training with Back-propagation



Training a CNN uses back-propagation and works the same as with an MLP:

- Step 1: Initialise all filters/weights with random values
- Step 2: Select a batch size, and feed-forward the training instances
- Step 3: Calculate the total error at the output layer
- Step 4: Use back-propagation to calculate the gradients of the error with respect to all weights in the network and use gradient descent to update all filter values / weights.
- Step 5: Repeat until the error is suitably minimised

CNN Building-Blocks



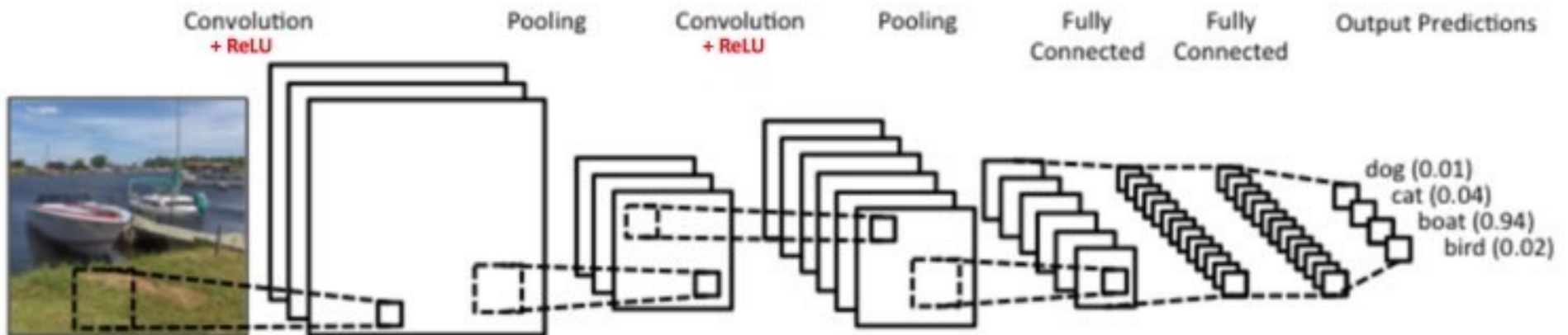
Now we have learned about some main building blocks of CNNs

- Convolutions (2D)
- Non-Linearity (Activations)
- Pooling (or sub-sampling)
- Fully Connected MLP

LeNet Architecture (1990s)

The LeNet is considered the classic CNN architecture that demonstrated a successful Deep network design for image recognition.

It contains a “relatively simple” combination of the 4 key CNN components



Architecture (Modified)



Define
Structure

```
model_cnn = Sequential()
model_cnn.add(Conv2D(32, (3, 3), kernel_regularizer=regularizers.l2(0.001), input_shape=(28, 28, 3)))
model_cnn.add(Activation('relu'))
model_cnn.add(MaxPooling2D(pool_size=(2, 2)))

model_cnn.add(Conv2D(64, (3, 3)))
model_cnn.add(Activation('relu'))
model_cnn.add(MaxPooling2D(pool_size=(2, 2)))

model_cnn.add(Flatten()) # this converts our 3D feature maps to 1D feature vectors
model_cnn.add(Dense(64))
model_cnn.add(Activation('relu'))
model_cnn.add(Dropout(0.5))
model_cnn.add(Dense(4))
model_cnn.add(Activation('softmax'))
```

Initialize
Computation
graph

```
sgd = optimizers.SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model_cnn.compile(loss='categorical_crossentropy',
                  optimizer=sgd,
                  metrics=[categorical_accuracy])
```

Popular CNN Architectures(Classification)



Convolutional Neural Networks have been around since early 1990s. Some influential architectures are:

LeNet(1990s)

One of the very first CNNconvolutional neural networks.

AlexNet (2012)

A deeper and much wider version of the LeNet Won, by a large margin, the difficult ImageNetLarge Scale Visual Recognition Challenge (ILSVRC) in 2012

It was a significant breakthrough with respect to the previous approaches and the current CNNs approaches can be attributed to this work.

ZF Net (2013)

The ILSVRC 2013 winner

An improvement on AlexNetby tweaking the architecture hyper-parameters.

Popular CNN (Classification)



- **GoogLeNet (2014)**

- ILSVRC 2014 winner

- Developed the Inception Module that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M).

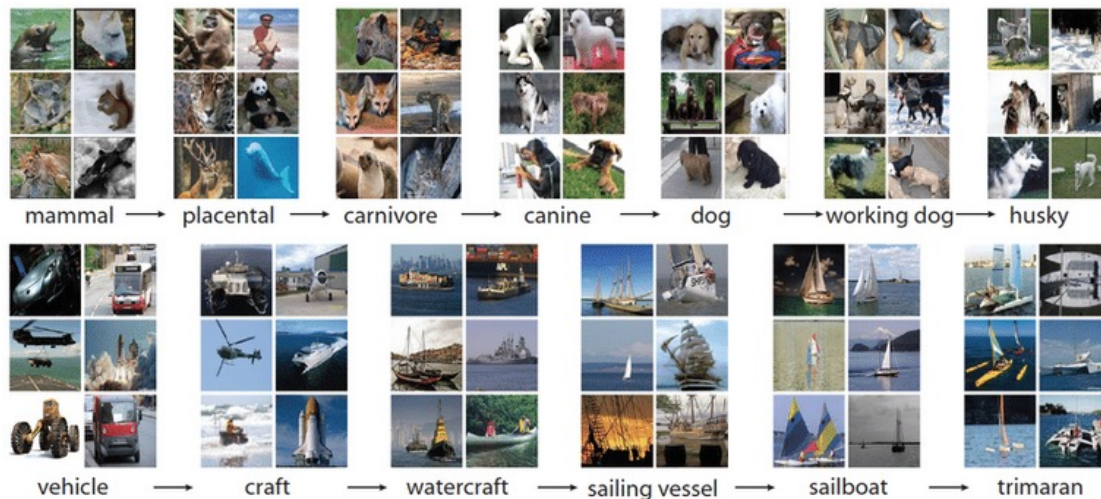
- **VGGNet (2014)**

- ILSVRC 2014 Runner-up Showed that the depth of the network (number of layers) is a critical component for good performance.

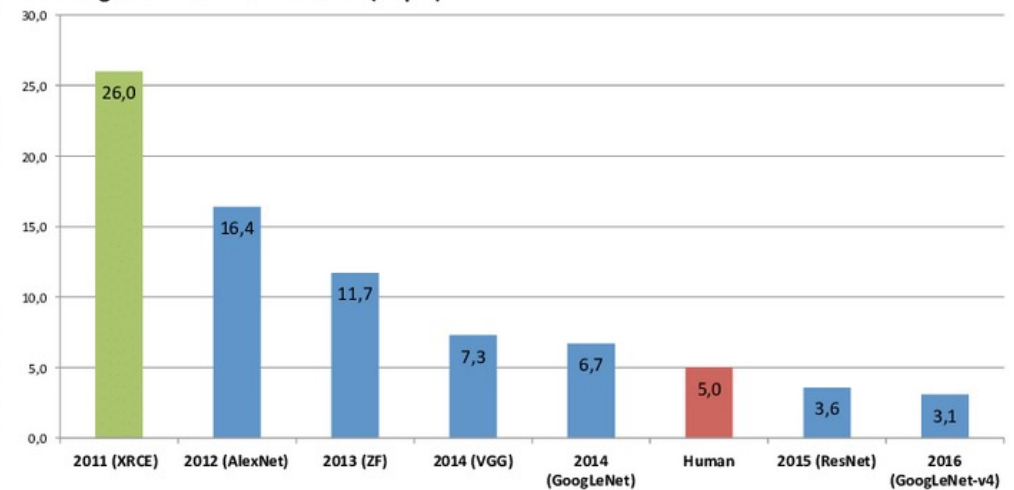
- **Others include ResNets(2015) & DenseNet(2016)**

- And many, many more since then

ImageNet: Large Scale Visual Recognition Challenge



ImageNet Classification Error (Top 5)



<http://www.image-net.org/challenges/LSVRC/>

More things not cover



- Activation functions
- CNN is not limited to image classification, can be used in: image segmentation, tracking, object detection, image captioning etc.

The preferred architectures change depending on the task.

- The CNN networks are also used to model speech and other types of tasks.
- Time series is modeled using recurrent neural networks (RNN) and LSTM.
- Unsupervised learning

Auto-Encoders and Generative Adversarial Networks (GANs)

Readings and Acknowledgement



1.The following website provides a good explanation on CNN:

<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>

2.Andrew Ng's course on CNN at coursera:

<https://www.coursera.org/learn/convolutional-neural-networks>

3.A few slides were from the following source:

<https://www.slideshare.net/LuMa921/deep-learning-a-visual-introduction>

Further readings:

- Ian Goodfellow's DL book, chapter 9 on CNN:

<https://www.deeplearningbook.org/>

- 12 Frequently Asked Questions on Deep Learning:

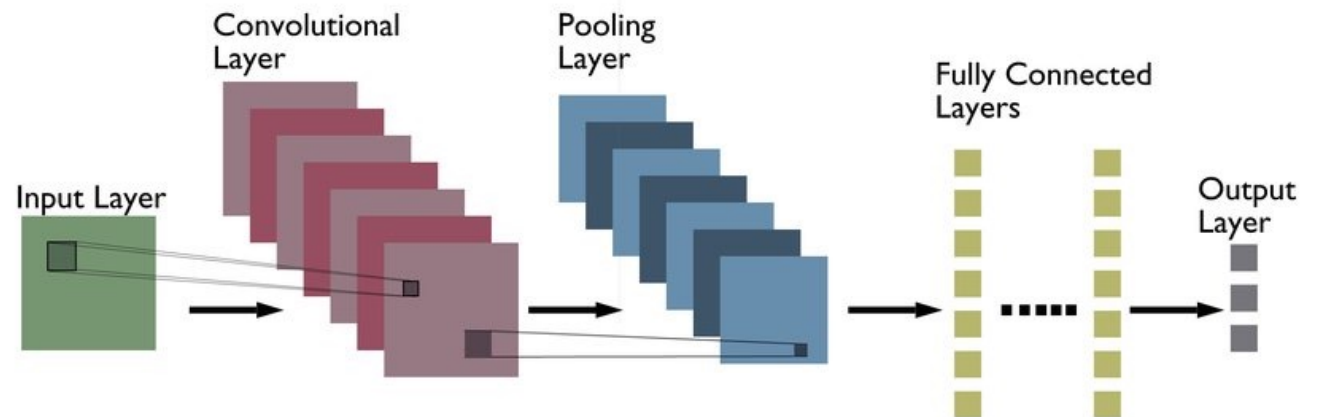
<https://www.analyticsvidhya.com/blog/2018/05/deep-learning-faq/>

Summary



Convolutional Neural Networks (CNNs)

- Convolutions
- Pooling
- Fully-connected MLP
- Strengths & Weaknesses
- Example CNNs



Thank you!